Relatório Prático: Kernel Mínimo em RISC-V
Boot, Comunicação Serial e Kernel Panic

João Pedro Sant Ana de Carvalho

[1]Faculdade de Computação – Universidade Federal de Mato Grosso do Sul (UFMS)
Campo Grande – MS – Brasil

santc.joao@gmail.com

Abstract. *This report documents the implementation of a minimal 32-bit RISC-V operating system kernel for the Operating Systems course. The completed tasks include kernel boot with a dedicated linker script (Task 1), serial communication via OpenSBI calls (Task 2), and a kernel panic mechanism for error handling (Task 4). Tasks 3 (full freestanding C library) and 5 (trap handling) were not implemented in this version. The implemented code was verified on the QEMU emulator.*

Resumo. *Este relatório apresenta o desenvolvimento de um kernel mínimo para a arquitetura RISC-V de 32 bits, realizado como atividade prática na disciplina de Sistemas Operacionais. O texto foca nas etapas que foram concluídas: o boot do kernel com linker script próprio (Trabalho 01), a saída serial via chamadas ao OpenSBI (Trabalho 02) e a implementação de kernel panic para lidar com falhas (Trabalho 04). Fica registrado que os Trabalhos 03 (biblioteca C básica completa) e 05 (tratamento de exceções) não foram realizados nesta entrega. O funcionamento dos módulos concluídos foi testado no emulador QEMU.*

1. Introdução

O objetivo deste projeto prático é criar um kernel básico do zero (bare-metal) para a arquitetura RISC-V de 32 bits (RV32), rodando no emulador QEMU junto com o firmware OpenSBI.

Das cinco tarefas propostas na disciplina, este relatório aborda as que foram finalizadas e testadas no repositório:

- **Trabalho 01**: Boot do kernel e linker script;
- **Trabalho 02**: Hello World com chamadas ao OpenSBI;
- **Trabalho 04**: Mecanismo de Kernel Panic.

Por restrições de tempo e escopo, os **Trabalhos 03** (biblioteca C com funções de string e memória) e **05** (tratador de exceções da CPU) **não foram realizados** nesta entrega.

2. Atividades Realizadas

2.1. Trabalho 01: Boot do Kernel e Linker Script

A primeira etapa foi definir como o kernel é organizado na memória por meio do linker script (`kernel.ld`). O endereço base foi colocado em `0x80200000`, que é o ponto padrão para onde o OpenSBI pula depois de inicializar o hardware.

O linker script divide a imagem nas seções usuais: `.text`, `.rodata`, `.data` e `.bss`. A seção `.text.boot` foi colocada logo no começo usando `KEEP` para garantir que o código de boot fique no endereço inicial. Logo após a seção `.bss` (delimitada pelos símbolos `__bss` e `__bss_end`), foram reservados 128 KB para a pilha do kernel, com o símbolo `__stack_top` marcando o topo da pilha.

No arquivo `kernel.c`, a função `boot` foi escrita em assembly inline com o atributo `naked`, evitando que o compilador adicione instruções extras de início ou fim de função. Essa função apenas ajusta o registrador de pilha (`sp`) para `__stack_top` e pula para `kernel_main`:

```
__attribute__((section(".text.boot")))
__attribute__((naked)) void boot(void) {
    __asm__ __volatile__(
        "mv sp, %[stack_top]\n"
        "j kernel_main\n"
        : : [stack_top] "r"(__stack_top)
    );
}
```

Ao entrar em `kernel_main`, a primeira ação é limpar a seção `.bss`, preenchendo todos os seus bytes com zero através de um `memset` local. Isso garante que variáveis globais e estáticas não comecem com valores aleatórios da memória.

2.2. Trabalho 02: Saída Serial com OpenSBI

Para conseguir imprimir mensagens no terminal do QEMU sem drivers complexos, utilizou-se a Supervisor Binary Interface (SBI) fornecida pelo OpenSBI através da instrução `ecall`.

A função `sbi_call` organiza os parâmetros nos registradores da CPU: `a7` recebe o Extension ID (EID), `a6` o Function ID (FID), `a0` a `a5` levam os argumentos, e o retorno vem em `a0` e `a1` (guardados na estrutura `struct sbiret`). Com isso, foi implementada a função `putchar` usando a extensão clássica *Console Putchar* (EID 1, FID 0):

```
void putchar(char ch) {
    sbi_call(ch, 0, 0, 0, 0, 0, 0, 1);
}
```

A partir do `putchar`, foi criada uma rotina simples para percorrer strings terminadas em `'\0'` e enviar caractere por caractere para a serial, permitindo imprimir mensagens como `"Hello World!"`.

2.3. Trabalho 04: Mecanismo de Kernel Panic

O quarto trabalho teve como foco criar uma forma simples de interromper o sistema quando acontece um erro fatal irrecuperável. No arquivo `kernel.h`, foi definida a macro `PANIC`:

```
#define PANIC(fmt, ...) \
    do { \
```

```
        printf("PANIC: %s:%d: " fmt "\n", \
               __FILE__, __LINE__, ##__VA_ARGS__); \
        while (1) {} \
    } while (0)
```

A macro usa o padrão `do { ... } while(0)` para funcionar bem dentro de condicionais (`if/else`). Foram utilizadas as macros do compilador `__FILE__` e `__LINE__` para mostrar no terminal o arquivo e a linha onde a falha aconteceu, além da mensagem passada como parâmetro. Depois de imprimir, ela entra em um loop infinito (`while(1)`), travando a CPU e impedindo que o kernel continue executando instruções com o estado corrompido.

3. Atividades Não Realizadas

Para fins de transparência sobre o escopo entregue, detalha-se o que ficou pendente:

- **Trabalho 03 (Biblioteca C Básica):** A proposta original pedia a implementação de funções de manipulação de memória e strings (`memcpy`, `strcpy`, `strcmp`), além de tipos próprios. No código atual, foi implementado apenas um `memset` enxuto direto no `kernel.c` e um `printf` básico em `common.c` para viabilizar a impressão do pânico. A biblioteca completa planejada em `common.c` e `common.h` não foi finalizada.
- **Trabalho 05 (Tratamento de Exceções):** Esta etapa previa a configuração do registrador `stvec`, a rotina `kernel_entry` em assembly para salvar todos os registradores na pilha (estrutura `trap_frame`), e a leitura dos registradores CSR (`scause`, `stval`, `sepc`) para identificar erros de hardware e instruções ilegais. Esse tratador não foi implementado, portanto o kernel ainda não captura exceções da CPU.

4. Resultados e Validação

O kernel foi compilado com o Clang/LLD para RISC-V 32-bit em modo *freestanding* e testado no QEMU através do script `run.sh`.

Os testes confirmaram o funcionamento das partes implementadas:

1. **Boot e Pilha:** A inspeção dos símbolos gerados no `kernel.map` e no desassemblador confirmou que o ponto de entrada `boot` foi colocado exatamente em `0x80200000` e a pilha começou em `0x80220018`.
2. **Saída Serial e Kernel Panic:** No `kernel_main`, executou-se intencionalmente a chamada `PANIC("booted!")` para testar a parada de emergência:
   `PANIC: kernel.c:42: booted!`
   A mensagem apareceu no terminal indicando corretamente o arquivo e a linha, e a linha seguinte (`printf("unreachable here!\n")`) não chegou a ser executada, confirmando que a CPU realmente travou no loop infinito.

5. Conclusão

Neste trabalho, foi possível colocar um kernel RISC-V para rodar do zero em bare-metal, estabelecer a comunicação serial com o OpenSBI para exibição no terminal e criar um mecanismo de kernel panic para contenção de erros. Os Trabalhos 03 e 05 não foram concluídos nesta entrega e ficam como etapas necessárias antes de avançar para tópicos como paginação de memória e escalonamento de processos.

References

[1] Nuta, S. (2023). *Operating System in 1,000 Lines*.

[2] RISC-V SBI Task Group (2022). *RISC-V Supervisor Binary Interface Specification*.